

Paradygmaty Programowania.

Laboratorium nr 5 (Przykładowe zastosowanie architektury MVC)

Michał Obara, Grzegorz Nieradka

Warszawa, Maj 2026

Zadanie 1. (2 pkt)

Pobrać pliki: `app.py`, `model.py`, `widok.py`, `kontroler.py`. Następnie uruchomić aplikację: `python3 app.py` i zapoznać się z efektami jej działania.

Uwaga! Ze względu na `import` zależności pliki muszą być w tym samym folderze, z którego jest uruchamiana aplikacja.

Stosując metody inżynierii wstecznej (w tym przypadku zdrowy rozsądek, zgromadzoną wiedzę i dostęp do kodu źródłowego) narysować diagram UML zaimplementowanego systemu.

Zastanowić się i odpowiedzieć na pytanie: Zarząd banku decyduje o likwidacji placówek stacjonarnych i przeniesieniu całej obsługi kredytowej do aplikacji mobilnej. Które klasy musiałyby zostać napisane całkowicie od nowa, a które można by pozostawić bez zmian?

Zarówno diagram klas UML jak i odpowiedź na pytanie umieścić w sprawozdaniu.

Zadanie 2. (2 pkt)

Zmodyfikować program wg. następujących wymagań:

1. Zmiana wymagań biznesowych. Zaostrożenie kryteriów ryzyka.
Dyrektor ryzyka nakazuje natychmiastowe zmiany, w związku z nową sytuacją rynkową.
W nowym algorytmie miesięczny dochód musi stanowić minimum 10% wnioskowanej kwoty (zmiana z 0.05 na 0.10). Należy również dodać twardy limit: maksymalna kwota kredytu to 200 000 zł. Wszystkie wnioski powyżej tej kwoty system ma odrzucić (`return False`).
2. Zmiana prezentacji danych. Zgodność z RODO.
Inspektor Ochrony Danych (IOD) nakazuje zabezpieczenie danych klientów na ekranach pracowników. Należy zmienić sposób prezentacji danych tak, aby system nie wyświetlał pełnego nazwiska klienta w rejestrze zaakceptowanych wniosków (opcja 2). W tym zadaniu wystarczy zmodyfikować wypisywanie nazwiska przez dodanie przed i po nim symbolu gwiazdki (np. `"" + w.name + ""`) i/lub zmianę etykiety na „Identyfikator Klienta”.
3. Modyfikacja kontrolera. Logowanie procedur bezpieczeństwa.
Szef ochrony operacyjnej wprowadza nową procedurę przy opuszczaniu stanowiska kasowego. Należy zmienić zachowanie metody `weryfikuj_stanowisko`. Trzeba tak ją zmodyfikować aby na ekranie było wypisywane ostrzeżenie: `\n [BEZPIECZEŃSTWO]: UWAGA! Pamiętaj o zabraniu karty pracowniczej z czytnika biometrycznego!`

W sprawozdaniu udokumentować wykonane zmiany, tzn. które linie, w których plikach i jak należy zmienić.

Zadanie 3. (2 pkt)

Bank planuje dodać do systemu funkcjonalność powiadomień SMS dla klientów po podjęciu decyzji kredytowej. Obecnie system nie posiada takich struktur. Bazując na opracowanym w Zadaniu 1 diagramie klas UML umieścić następujące zmiany strukturalne:

1. Zaprojektować nową klasę `SmsNotyfikator`
2. Klasa `SmsNotyfikator` powinna zawierać co najmniej jedną metodę:
`wyslij_sms(numer_telefonu, tresc)`.
3. Zmodyfikować klasę `BankController` tak, aby wykorzystywała wzorzec *kompozycji* (silnej relacji zawierania) z nową klasą `SmsNotyfikator`.
4. Oznaczyć tę relację na diagramie odpowiednią strzałką (diamentem) symbolizującą kompozycję i określić jej licznosc (krotnosc).

W sprawozdaniu umieścić zaktualizowany diagram klas UML oraz krótkie uzasadnienie (w 2-3 zdaniach), dlaczego relacja między `BankController` a `SmsNotyfikator` to kompozycja, a nie agregacja.

Biorąc pod uwagę architekturę MVC, uzasadnić, czy `SmsNotyfikator` powinien być częścią warstwy Widoku (View), czy jest to usługa infrastrukturalna wywoływana przez Kontroler (Controller)? Jak zmieniłaby się ta odpowiedź, gdyby SMS był wysyłany nie do klienta, a do wewnętrznego logu diagnostycznego systemu?
